

Resumo do Artigo: Object Constraint Language (OCL) — Um Guia Definitivo

Autores: Jordi Cabot e Martin Gogolla (2012)

Aluno: Nicolas Araújo Fonseca Pimenta

1. Introdução e Contexto Histórico

O artigo apresenta a OCL, ou Object Constraint Language, como uma linguagem textual que surgiu para complementar a UML, a Unified Modeling Language. Os autores explicam que a OCL foi criada em 1995 dentro da IBM e, em 1997, passou a fazer parte oficial da UML. O motivo da sua criação é simples: linguagens gráficas como a UML são ótimas para mostrar a estrutura de um sistema, ou seja, classes, atributos e relacionamentos, mas têm limitações para expressar regras detalhadas e específicas. Por exemplo, um diagrama de classes consegue mostrar que um cliente faz aluguéis de carros, mas não consegue dizer, por si só, que “um cliente na lista negra não pode alugar carros”. É exatamente nessas situações que a OCL entra em cena, oferecendo uma forma precisa e formal de escrever essas regras adicionais.

2. Principais Características da OCL

A OCL é descrita pelos autores como uma linguagem com quatro características fundamentais que a tornam especial e adequada para a modelagem de software:

- **Tipada:** toda expressão escrita em OCL tem um tipo definido, como inteiro, texto, booleano ou uma classe, e precisa seguir as regras desse tipo. Isso ajuda a evitar erros lógicos no momento da escrita das regras.
- **Declarativa:** a linguagem descreve “o que” deve acontecer ou ser verdadeiro, e não “como” fazer. Isso significa que ela não tem comandos de atribuição como em linguagens de programação tradicionais, como C ou Java.
- **Sem efeitos colaterais:** as expressões em OCL podem consultar e verificar o estado do sistema, mas nunca modificá-lo. Ou seja, a OCL serve para descrever e validar, não para alterar dados.
- **De especificação:** ela apenas define regras e condições, sem dizer como o sistema deve ser implementado tecnicamente. Isso a torna independente de qualquer linguagem de programação.

3. Tipos de Expressões que a OCL Permite Criar

Os autores destacam cinco tipos principais de expressões que podem ser escritas em OCL, cada uma com uma finalidade diferente dentro da modelagem do sistema:

- **Invariantes:** são as expressões mais comuns. Funcionam como regras que devem ser sempre verdadeiras para todas as instâncias de uma classe. Um exemplo simples seria: “o valor de uma cotação deve ser sempre maior que zero”. Um exemplo mais complexo, dado no artigo, é a regra de que pessoas na lista negra não podem ter aluguéis ativos.

- **Inicialização de propriedades:** permite definir o valor inicial de um atributo no momento em que um objeto é criado. Por exemplo, todo cliente novo começa com o atributo “premium” igual a falso.
- **Regras de derivação:** definem como o valor de um atributo derivado deve ser calculado automaticamente a partir de outros dados. O exemplo do artigo mostra como o desconto de um cliente pode ser calculado com base em ele ser premium ou não, e em quantos carros de categoria alta ele já alugou.
- **Operações de consulta:** são expressões usadas para fazer perguntas ao sistema e obter informações sem alterar nada. Por exemplo, perguntar se um determinado carro é o mais alugado da empresa.
- **Contratos de operação:** definem condições antes, chamadas de pré-condições, e depois, chamadas de pós-condições, da execução de uma operação. Servem para garantir que a operação só seja executada se determinadas condições forem cumpridas e produzem o resultado esperado ao final.

4. Sistema de Tipos e Coleções da OCL

Uma parte importante do artigo é dedicada a explicar o sistema de tipos da OCL. A linguagem trabalha com tipos básicos, como Integer, Real, String e Boolean, tipos definidos pelo usuário, como classes e enumerações, e os chamados tipos templates, que servem para construir coleções de elementos. Os quatro tipos de coleção são:

- **Set (conjunto):** uma coleção sem ordem definida e sem elementos repetidos. A ordem em que os elementos são adicionados não importa.
- **Bag (saco):** também não tem ordem, mas permite elementos repetidos. Um exemplo prático seria contar quantas vezes o mesmo carro foi alugado.
- **Sequence (sequência):** tem ordem e permite repetições. É como uma lista, em que a ordem dos itens importa.
- **OrderedSet (conjunto ordenado):** tem ordem, mas não permite repetições. Útil quando se precisa manter uma sequência única de elementos.

Além disso, a OCL oferece operações poderosas para manipular essas coleções, como **select**, que filtra elementos, **collect**, que transforma elementos, **forAll**, que verifica uma condição em todos os elementos, e **exists**, que verifica se existe pelo menos um elemento que atende a uma condição, entre outras. A linguagem também trabalha com lógica de três valores, ou seja, verdadeiro, falso e nulo, parecida com a usada em bancos de dados SQL.

5. Ferramentas de Apoio à OCL

Os autores apresentam uma lista das principais ferramentas que dão suporte à OCL, organizadas em quatro categorias:

- **Parsers e IDEs:** ferramentas como o MDT/OCL, do Eclipse, e o DresdenOCL servem para interpretar e analisar as expressões escritas em OCL, verificando se estão corretas do ponto de vista da sintaxe.

- **Ferramentas UML com suporte à OCL:** softwares como ArgoUML, Rational Rose, Enterprise Architect, MagicDraw e Borland Together permitem escrever expressões OCL diretamente em modelos UML. Os autores comentam, porém, que poucos softwares UML oferecem suporte completo à OCL.
- **Ferramentas de validação e verificação:** como a USE, sigla para UML-based Specification Environment, que permite simular o sistema e verificar se as regras OCL fazem sentido. Isso ajuda a identificar problemas como modelos com regras demais, que impedem situações válidas, ou com regras de menos, que permitem situações inválidas.
- **Geração de código a partir de OCL:** algumas ferramentas conseguem transformar expressões OCL em código real, geralmente como gatilhos de banco de dados, conhecidos como triggers SQL, ou como verificações dentro de métodos em linguagens orientadas a objetos.

6. Desafios e Agenda de Pesquisa para o Futuro

Apesar da importância da OCL, os autores reconhecem que a linguagem ainda tem desafios a serem superados. Eles apontam quatro áreas principais de pesquisa:

- **Modularização e extensibilidade:** a biblioteca padrão da OCL é grande e tem operadores que se sobrepõem, o que pode confundir iniciantes. Por outro lado, faltam funções importantes, como funções estatísticas básicas. Seria útil permitir que usuários criassem bibliotecas próprias e as importassem quando necessário.
- **Melhorias na linguagem:** alguns aspectos da sintaxe ainda causam confusão, como a diferença entre o uso do ponto e da seta para acessar coleções. Também faltam atalhos para escrever expressões mais curtas e uma definição completa da semântica formal da linguagem.
- **Análise e raciocínio eficientes:** quando os modelos são muito grandes, como em engenharia reversa de código, a OCL apresenta problemas de desempenho. Pesquisas estão sendo feitas para melhorar a avaliação incremental e a avaliação preguiçosa, conhecida como lazy, das expressões, e até mesmo para usar computação em nuvem para acelerar a análise.
- **Criação de uma comunidade ativa:** um dos pontos mais críticos é a falta de uma comunidade forte de praticantes e pesquisadores em torno da OCL. Os autores defendem que sem essa base ativa, a evolução da linguagem fica mais lenta e a adoção pelo mercado é prejudicada.

7. Conclusão

O artigo conclui que a OCL é uma linguagem fundamental dentro da Engenharia Dirigida por Modelos, conhecida como Model-Driven Engineering ou MDE, pois consegue expressar de forma precisa aquilo que diagramas como os da UML não conseguem. Ela é usada para escrever invariantes, regras de derivação, contratos de operação, transformações de modelos e até mesmo regras de boa formação para novas linguagens de domínio específico, conhecidas como DSLs. Apesar de não ser perfeita e de ainda enfrentar desafios, como a falta de uma comunidade forte e a necessidade de melhorias na linguagem e nas ferramentas, a OCL é considerada essencial para quem trabalha com

modelagem de software de forma séria. Os autores deixam claro que vale a pena investir tempo no aprendizado e no desenvolvimento contínuo dessa linguagem, dado o papel central que ela ocupa no paradigma de desenvolvimento orientado a modelos.